

Представление чисел в памяти компьютера Битовые операции

Память компьютера

- Оперативная, RAM: работает только пока компьютер включен. После выключения обнуляется. Быстрая
- Постоянная: диск (HDD), SSD: сохраняется и после выключения компьютера. Значительно медленнее RAM, но энергонезависима.
- Флешка – это какая память?

Память компьютера

- Будем говорить про RAM
- Память устроена очень сложно
- Бравые разработчики железа и ОС поддерживают для нас абстракцию
- Думать про память можно и нужно как про линейный набор ячеек

0	1	2	3	4	5	6	7	8	9	A	B	C	...
0xAA	0x07	0x00	0x09	0x09	0x01	0xFE	0xC8	0x77	0x00	0x00	0x00	0x07	...

Память компьютера

- Индекс (т.е. номер) ячейки памяти называется **адресом**
- Байт – наименьшая адресуемая единица информации
- Где на рисунке байты? Сколько их?
- Всегда ли байт = 8 бит?

0	1	2	3	4	5	6	7	8	9	A	B	C	...
0xAA	0x07	0x00	0x09	0x09	0x01	0xFE	0xC8	0x77	0x00	0x00	0x00	0x07	...

Память компьютера

- Из определения: байт не всегда 8 бит
- В компьютере PDP-8s 1 байт = 12 бит
- На современных компьютерах почти везде байт = 8 бит, но это не определение!
- На рисунке одна ячейка – две шестнадцатеричные цифры. Почему?

0	1	2	3	4	5	6	7	8	9	A	B	C	...
0xAA	0x07	0x00	0x09	0x09	0x01	0xFE	0xC8	0x77	0x00	0x00	0x00	0x07	...

Память компьютера

- На рисунке одна ячейка – две шестнадцатеричные цифры. Почему?
- Тут байт = 8 бит, т.е. может хранить от

0b00000000 до 0b11111111

0x00 до 0xFF

- Поэтому байт на *современных компьютерах* это две 16-ричные цифры

0	1	2	3	4	5	6	7	8	9	A	B	C	...
0xAA	0x07	0x00	0x09	0x09	0x01	0xFE	0xC8	0x77	0x00	0x00	0x00	0x07	...

Как числа хранятся в памяти

- Рассмотрим целые, положительные числа
- Как положить такое число в память?

Пусть мы хотим:

- по адресу 0x7 положить число 44
- по адресу 0x9 положить число 1032247

0	1	2	3	4	5	6	7	8	9	A	B	C	...
...	...	...	...	...	...	...	??	...	??	...	...	...	. . .

Как числа хранятся в памяти

Пусть мы хотим:

- по адресу 0x7 положить число 44

Тут все просто:

- $44_{10} = 0x2C$
- записываем 0x2C в ячейку 7

5	6	7	8	9	A	B	C	...
...	...	0x2C	...	??	...	...	...	...

Как числа хранятся в памяти

Пусть мы хотим:

- по адресу 0x9 положить число 1032247
- $1032247_{10} = 0xFC037$
- У нас три байта: 0xFC, 0xC0, 0x37
- Какой вариант верный?

8	9	A	B	C	...
...	0xFC	0xC0	0x37	...	...

8	9	A	B	C	...
...	0x37	0xC0	0xFC	...	...

Как числа хранятся в памяти

- $1032247_{10} = 0xFC037$
- У нас три байта: 0xFC, 0xC0, 0x37
- Какой вариант верный?
- Верны оба варианта!
- Сверху: big-endian: старший байт располагается по младшему адресу
- Снизу: little-endian: старший байт располагается по старшему адресу

big-endian

8	9	A	B	C	...
...	0xFC	0xC0	0x37	...	...

little-endian

8	9	A	B	C	...
...	0x37	0xC0	0xFC	...	...

Как числа хранятся в памяти

- Сверху: big-endian: старший байт располагается по младшему адресу
- Снизу: little-endian: старший байт располагается по старшему адресу
- Какой вариант логичнее и используется в современных компьютерах?

big-endian

8	9	A	B	C	...
...	0xFC	0xC0	0x37	...	. . .

little-endian

8	9	A	B	C	...
...	0x37	0xC0	0xFC	...	. . .

Задания

Записать в big-, little-endian

1. 59774123759

2. 4352359

3. 12192683

4. 11209477

Кодирование отрицательных чисел

- Пора задуматься про другие числа!
- Проблема: как представить знак числа?

Кодирование отрицательных чисел

- Проблема: как представить знак числа?
- Очевидная идея: выделить один бит, который будет указывать знак числа.

Кодирование отрицательных чисел: прямой код

Идея: старший бит хранит знак числа:

- “0” – положительное число
- “1” – отрицательное число

Оставшиеся биты хранят модуль числа.

Примеры*:

$$+5_{10} = 00000101_2$$

$$-5_{10} = 10000101_2$$

*здесь и далее в примерах рассматриваем 8-битные числа

Кодирование отрицательных чисел: прямой код

Идея: старший бит хранит знак числа:

- “0” – положительное число
- “1” – отрицательное число

Оставшиеся биты хранят модуль числа.

Какие проблемы возникают при таком представлении?

Кодирование отрицательных чисел: прямой код

Идея: старший бит хранит знак числа:

- “0” – положительное число
- “1” – отрицательное число

Оставшиеся биты хранят модуль числа.

Проблемы:

- Два нуля: $+0_{10} = 00000000_2$ и $-0_{10} = 10000000_2$
- Арифметические операции усложняются: знак и модуль приходится обрабатывать отдельно

Кодирование отрицательных чисел: обратный код

Идея: для получения отрицательного числа инвертируем все биты соответствующего положительного числа

Примеры:

$$+5_{10} = 00000101_2$$

$$-5_{10} = 11111010_2$$

Кодирование отрицательных чисел: обратный код

Идея: для получения отрицательного числа инвертируем все биты соответствующего положительного числа

Какие плюсы/минусы у такого представления?

Кодирование отрицательных чисел: обратный код

Идея: для получения отрицательного числа инвертируем все биты соответствующего положительного числа

Плюсы:

- Легко изменить знак числа
- Арифметические операции значительно проще, чем в прямом кодировании

Минусы:

- Все еще существует два представления нуля
- При сложении необходимо перенос из старшего бита прибавлять к младшему биту

Кодирование отрицательных чисел: дополнительный код

Идея: для получения отрицательного числа инвертируем все биты соответствующего положительного числа и прибавляем 1

Диапазон чисел, представимых с помощью N бит: $[-2^{N-1}; 2^{N-1} - 1]$

Примеры:

$$+5_{10} = 00000101_2$$

$$-5_{10} = 11111011_2$$

Кодирование отрицательных чисел: дополнительный код

Идея: для получения отрицательного числа инвертируем все биты соответствующего положительного числа и прибавляем 1

Диапазон чисел, представимых с помощью N бит: $[-2^{N-1}; 2^{N-1} - 1]$

Плюсы:

- Легко выполнять арифметические операции
- Существует только одно представление нуля

Кодирование отрицательных чисел: примеры

Необходимо перевести числа в выражении в дополнительный код, вычислить значение, перевести обратно:

1) $42 - 69$

2) $-3 - 12$

3) $-44 + 63$

4) $228 - 322$

Битовые операции

- Битовые операции это операции, которые выполняются независимо для каждой пары битов
- Каждую операцию можно представить таблицей истинности
- Таблица истинности – способ задания логического выражения или операции

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Битовые операции: and, or

- and, побитовое “и” &

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

- or, побитовое “или” |

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Битовые операции: xor, nand

- xor, “исключающее или” ($\wedge$) :

A	B	$A \wedge B$
0	0	0
0	1	1
1	0	1
1	1	0

- nand, “НЕ-И”:

A	B	$A \text{ nand } B$
0	0	1
0	1	1
1	0	1
1	1	0

Битовые операции: сдвиг

- Сдвиг влево ($\ll$) – все биты сдвигаются влево, младшие биты = 0

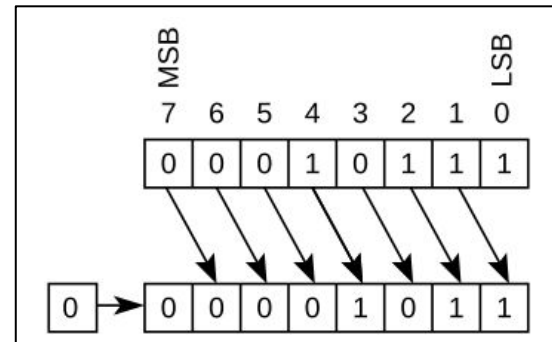
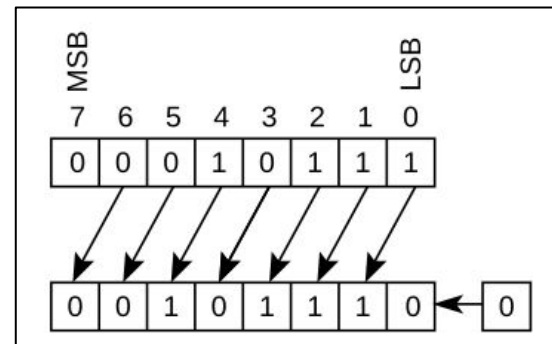
Пример: 0000 1011 $\ll$ 2 =
 0010 1100

 1111 1111 $\ll$ 3 =
 1111 1000

- Сдвиг вправо ($\gg$) – все биты сдвигаются вправо, старшие биты = 0

Пример: 0000 1011 $\gg$ 1 =
 0000 0101

 0000 0111 $\gg$ 3 =
 0000 0000

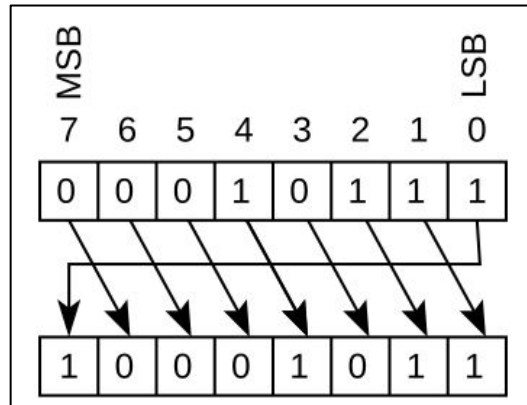
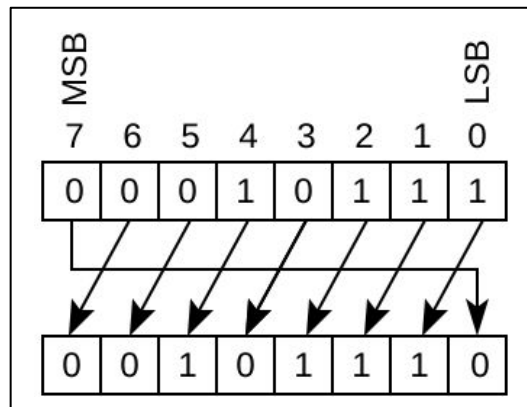


Циклические сдвиги

При циклическом сдвиге уходящий бит появляется на месте появившегося свободного на другом конце числа

Примеры:

- Влево на 1 бит: 1001 → 0011
- Вправо на 1 бит: 1001 → 1100



Битовые операции

- 1) 1011 0110 and 0110 1100
- 2) 1011 0110 or 0110 1100
- 3) 1011 0110 xor 0110 1100
- 4) 1011 0110 nand 0110 1100
- 5) 1011 0111 << 2
- 6) 1011 0111 >> 3
- 7) ROL(1011 0111, 1)
- 8) ROR(1011 0111, 4)

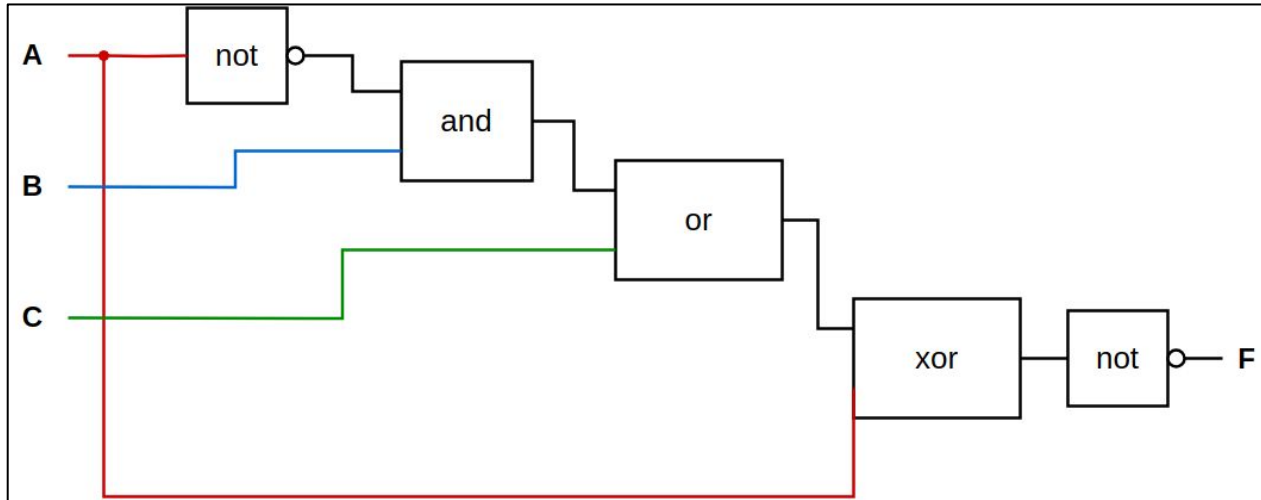
Битовые операции

- 1) $1011\ 0110 \text{ and } 0110\ 1100 = 0010\ 0100$
- 2) $1011\ 0110 \text{ or } 0110\ 1100 = 1111\ 1110$
- 3) $1011\ 0110 \text{ xor } 0110\ 1100 = 1101\ 1010$
- 4) $1011\ 0110 \text{ nand } 0110\ 1100 = 1101\ 1011$
- 5) $1011\ 0111 \ll 2 = 1101\ 1100$
- 6) $1011\ 0111 \gg 3 = 0001\ 0110$
- 7) $\text{ROL}(1011\ 0111, 1) = 0110\ 1111$
- 8) $\text{ROR}(1011\ 0111, 4) = 0111\ 1011$

Логические схемы

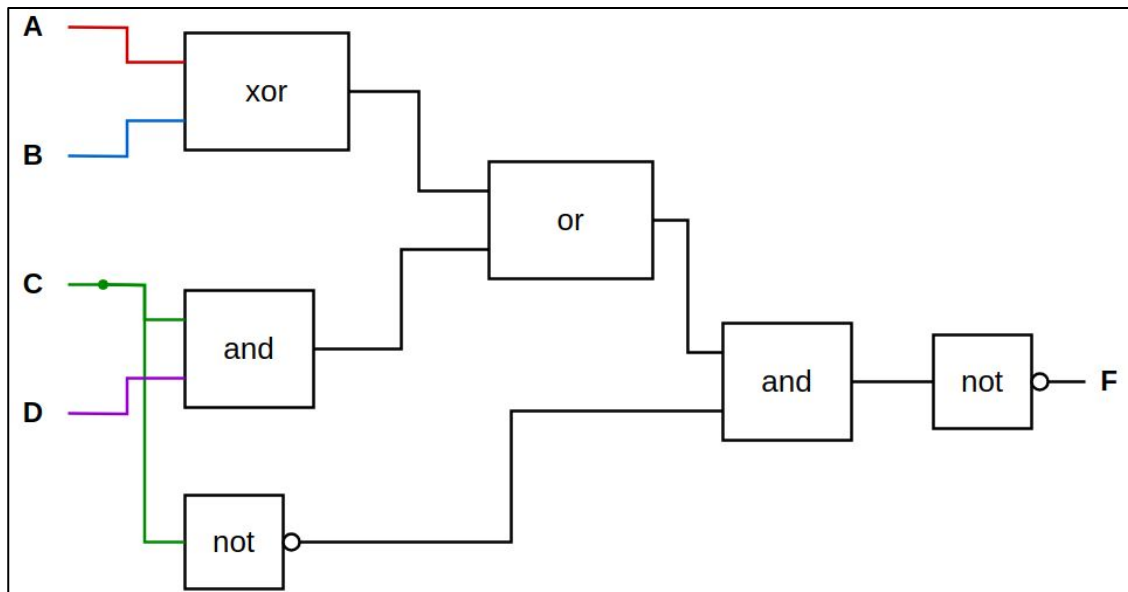
Любое выражение можно представить как схему. Тут все интуитивно

$$F(A, B, C) = \text{NOT} [(\text{NOT } A \text{ AND } B \text{ OR } C) \text{ XOR } A]$$



Логические схемы

Теперь сами



Рекап

- Что такое байт?
- Чем HDD отличается от RAM?
- Что такое endianness?
- Какие существуют способы кодирования отрицательных чисел?
- Что такое таблица истинности?
- Какие логические операции вы знаете?